Requirements Specification (SRS)/Design Document (DD) of [25-26J-065]

Project ID : 25-26J-065

Project Title : Multi-Modal Machine Learning Approach for Threat Detection in WhatsApp and Telegram: Spam Links, Document Based Malware, Chat Behavior, and Sensitive Content Analysis

Student Details:	
Student Registration Number	Student Name
IT22594440	G.W.A.P.Gangoda
IT22561220	H.K.I. Sewwandi
IT22560230	P.S.N.A.Rupasinghe
IT22602428	N.T.K.D Perera

Supervisor : Mr Deemantha Siriwardana

Date of Submission: 11/01/2026

Table of Contents

1.INTRODUCTION 3
1.1 PURPOSE 3
1.2 SCOPE 4
1.3 Definitions, Acronyms, and Abbreviations 5
1.4 Overview 6
2. Overall Descriptions 8
2.1 Product perspective 10
2.1.1 System interfaces 11
2.1.2 User interfaces 12
2.1.3 Hardware interfaces 17
2.1.4 Software interfaces 18
2.1.5 Communication interfaces 19
2.1.6 Memory constraints 20
2.1.7 Operations 21
2.1.8 Site Adaptation Requirements 22
2.2 Product functions 23
2.3 User characteristics 25
2.4 Constraints 27
2.5 Assumptions and dependencies 29
2.6 Apportioning of requirements 31
3. Specific requirements(1) (for Software Dev. Oriented Projects - SRS) 33
3.1 External interface requirements 33
3.1.1 User interfaces 33
3.1.2 Hardware interfaces 34
3.1.3 Software interfaces 35
3.1.4 Communication interface 36
3.2 Classes/Objects < For Software Dev. Oriented Projects> 37
3.3 Performance requirements 38
3.4 Design constraints 40
3.5 Software system attributes 42
3.5.1 Reliability 42
3.5.2 Availability 42
3.5.3 Security 42
3.5.4 Maintainability 43
3.6 Other Requirements 44
4. References 46

1.INTRODUCTION

1.1 PURPOSE

The purpose of this Software Requirements Specification (SRS) document is to provide a complete, precise, and unambiguous description of the functional and non-functional requirements of the proposed Multi-Modal Machine Learning–Based Threat Detection System for WhatsApp and Telegram. This document establishes a structured representation of system expectations, constraints, interfaces, and quality attributes required for successful development and deployment.

This SRS is intended to serve as a primary reference document for all stakeholders involved in the project, including academic supervisors, system developers, evaluators, testers, and future maintainers. By defining system behavior and operational boundaries in a standardized manner, the document ensures a shared understanding of system objectives and reduces the risk of misinterpretation or scope deviation during development.

The document outlines what the system shall do, including threat detection capabilities, user interaction requirements, server-assisted analysis constraints, and privacy-preserving operations, while deliberately excluding implementation-level details such as algorithms, programming languages, or platform-specific design decisions. This separation allows design and implementation choices to evolve without affecting the approved requirements.
Additionally, this SRS supports the entire software development lifecycle by acting as:

- A foundation for system architecture and detailed design
- A baseline for verification and validation activities
- A reference for performance evaluation and security assessment
- A guide for future enhancements, scalability planning, and maintenance

Overall, this document ensures that the proposed system meets its intended goals of enhancing user safety, preserving privacy, and enabling secure threat detection within encrypted messaging environments, while remaining adaptable to future technological and regulatory changes.

1.2 SCOPE

This document covers the requirements of a mobile-based security application designed to protect users—especially children—from multiple cyber threats encountered through WhatsApp and Telegram.

The scope of this SRS includes requirements related to the following integrated system aspects:

- Detection and classification of spam and malicious URLs shared in chats using sandbox-based analysis and machine learning
- Identification of malware-infected documents (PDF, DOCX, EXE) through sandbox execution and behavioral analysis
- Analysis of chat behavior and emotional patterns to identify fake, manipulative, or suspicious user accounts
- Detection of sensitive content (nudity, explicit images/videos, identity documents, credit card details) using lightweight on-device ML models
- Generation of real-time alerts and notifications
- Provision of a parental monitoring dashboard for visibility and control
- Ensuring privacy preservation, mobile performance efficiency, and compatibility with encrypted messaging environments

This SRS does not cover internal implementation details such as algorithm tuning, training datasets in detail, or platform-specific source code.

Term / Acronym	Description
ML	Machine Learning
AI	Artificial Intelligence
NLP	Natural Language Processing
CV	Computer Vision
URL	Uniform Resource Locator
Sandbox	Isolated execution environment used to safely analyze files or links
E2EE	End-to-End Encryption
WhatsApp	Encrypted instant messaging platform
Telegram	Cloud-based messaging platform
NSFW	Not Safe For Work (explicit or sensitive media)
CNN	Convolutional Neural Network
OCR	Optical Character Recognition
TP	True Positive
FP	False Positive
FN	False Negative
TN	True Negative
API	Application Programming Interface
UI	User Interface
UX	User Experience
APK	Android Package Kit
TFLite	TensorFlow Lite
LLM	Large Language Model
NIC	National Identity Card

1.3 Definitions, Acronyms, and Abbreviations

1.4 Overview

The proposed system is a multi-modal mobile security application designed to enhance online safety on WhatsApp and Telegram, with a strong focus on protecting children and non-technical users.

The software integrates machine learning, sandboxing, computer vision, and natural language processing to analyze multiple threat vectors within messaging platforms while respecting encryption and user privacy.

Main Goals

- Prevent exposure to malicious links, infected documents, sensitive media, and fake users
- Provide real-time, privacy-preserving detection directly on mobile devices
- Support parental supervision without intrusive monitoring
- Balance security, usability, and mobile performance

Core System Components

1. **URL Threat Detection Module**
   Analyzes suspicious links using static features and sandbox-based dynamic behavior to classify them as safe, spam, or malicious.
2. **Document Malware Detection Module**
   Executes shared documents in a controlled sandbox environment and uses ML to detect malicious behavior patterns.
3. **Chat Behavior & Emotion Analysis Module**
   Uses NLP and behavioral analytics to detect emotional manipulation, fake accounts, and suspicious interaction patterns.
4. **Sensitive Content Detection Module**
   Scans images and videos before sending to detect nudity, explicit content, identity documents, and financial details using lightweight on-device models.

Target Users

- Children and teenage users of WhatsApp and Telegram
- Parents or guardians monitoring online safety
- General mobile users seeking enhanced messaging security

System Characteristics

- Mobile-first and lightweight
- Fully privacy-aware with on-device processing
- Compatible with encrypted messaging constraints
- Designed for real-time detection and alerts

2. Overall Descriptions

The proposed system is a mobile-based, multi-modal threat detection application designed to enhance user safety on encrypted messaging platforms such as WhatsApp and Telegram, with particular emphasis on protecting children and non-technical users from emerging cyber threats. The system integrates multiple analytical techniques, including machine learning, sandbox-based dynamic analysis, computer vision, and natural language processing, to detect and mitigate diverse threat vectors encountered in modern messaging environments.

The software adopts a hybrid operational model, where sensitive and potentially harmful artifacts—such as suspicious URLs and shared documents—are analyzed within a secure server-side sandbox environment, while privacy-sensitive and latency-critical operations—such as image and video content analysis—are performed directly on the user's device. This approach ensures that malware is never executed on the user's device while maintaining real-time responsiveness and data confidentiality.

Unlike traditional mobile security applications that rely on signature-based detection or centralized cloud scanning, the proposed system emphasizes context-aware and behavior-based detection. It analyzes URLs for phishing, redirection, and malicious behavior; inspects documents for embedded exploits or payloads; evaluates media content for sensitive or explicit material; and examines chat interaction patterns to identify emotional manipulation or suspicious behavior. These capabilities enable the system to detect both known and previously unseen threats.

The system is designed using a modular and extensible architecture, allowing individual components—such as URL analysis, document malware detection, sensitive content detection, and behavioral analytics—to evolve independently. This modularity supports future enhancements, including the integration of improved machine learning models, additional messaging platforms, new threat categories, and advanced parental control features, without requiring major architectural redesign.

From a user perspective, the system prioritizes non-intrusive protection and informed decision-

making rather than restrictive control. Users are provided with clear, real-time alerts and risk explanations, while parents or guardians are offered summarized insights and monitoring capabilities through a secure dashboard. All operations are conducted in a privacy-preserving manner, respecting the constraints of end-to-end encryption and minimizing data collection, storage, and transmission.

Overall, the proposed system aims to deliver a scalable, secure, and user-centric threat detection solution that balances effectiveness, usability, and ethical considerations, making it suitable for long-term deployment in rapidly evolving digital communication environments.

2.1 Product perspective

The proposed software can be categorized as a mobile security and parental-assistance application tailored specifically for encrypted messaging platforms.

Comparison with Existing Solutions

- Traditional antivirus applications primarily rely on signature-based detection and are ineffective against zero-day threats, obfuscated URLs, and social engineering attacks.
- Cloud-based content moderation systems raise privacy concerns and are unsuitable for end-to-end encrypted platforms.
- Standalone parental control apps often operate at device or network level, lack real-time messaging awareness, and provide limited insight into chat-based threats.
- Blacklist-based URL scanners fail to detect newly generated or dynamically redirected links.

In contrast, the proposed system:

- Combines URL analysis, document malware detection, chat behavior analysis, and sensitive media detection into a single application.
- Uses sandbox-based dynamic analysis and lightweight ML models optimized for mobile devices.
- Operates within the constraints of encrypted messaging, without breaking end-to-end encryption.
- Provides context-aware alerts and parental dashboards instead of blocking communication entirely.

2.1.1 System interfaces

The proposed system interacts with multiple internal and external systems to perform threat detection and reporting while preserving end-to-end encryption constraints.
The primary system interfaces include:

- **Mobile Operating System Interface**

  The application interfaces with the Android operating system to access media files, document storage, notifications, background services, and network connectivity using OS-level APIs and permission mechanisms.

- **Messaging Platform Interaction Interface**

  The system interacts indirectly with WhatsApp and Telegram through user-initiated file sharing, accessibility services, and content selection workflows without accessing encrypted message payloads or modifying platform behavior.

- **Backend Server Interface**

  The mobile application communicates with a secure backend server for server-assisted analysis of URLs and documents. Only extracted features and metadata are transmitted, not raw personal content.

- **Sandbox Execution Environment Interface**

  The backend server interfaces with isolated sandbox environments to safely execute suspicious URLs and documents for behavioral analysis.

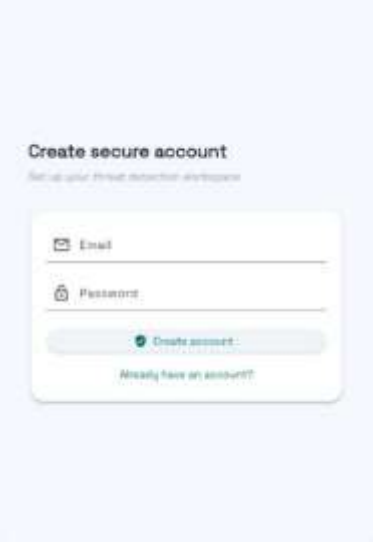
- **Parental Dashboard Interface**

  Authorized users access summarized threat reports and alerts through a secure dashboard interface connected to the backend server.

2.1.2 User interfaces

Login Interface

This is the first screen you see when starting the app for the first time. You type your email and create a strong password. Then you press the "Create account" button. It sets up your personal threat detection workspace. If you already have an account, there is a small link to go to login instead.



This screen lets you enter the app if you already made an account. You can type your email and password, or use the fast "Continue with Google" button. There is also a "Forgot password?" link if you cannot remember your password. The goal is to sign in safely to see your security information.

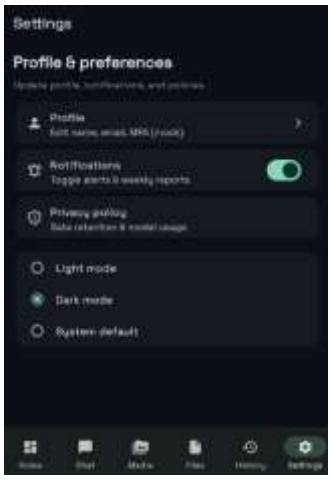
Dashboard Interface

This is the main home screen that shows how safe your digital life is right now. It has four colored cards that tell you: how many bad/malicious things were found, how many suspicious images or media, the emotional feeling of your chats (good or bad), and how many dangerous items are blocked/quarantined. It also shows small changes like "+3 today" so you know if things are getting better or worse.

This part of the dashboard lets you start scanning immediately. There are three big buttons: Scan chats (text/messages), Scan media (photos/videos), and Scan files. Below them you see the last few automatic or manual scans with date, what was checked, and risk level (LOW = green, MEDIUM = orange, HIGH = red). It is for quick checks and seeing recent results.

Setting Interface

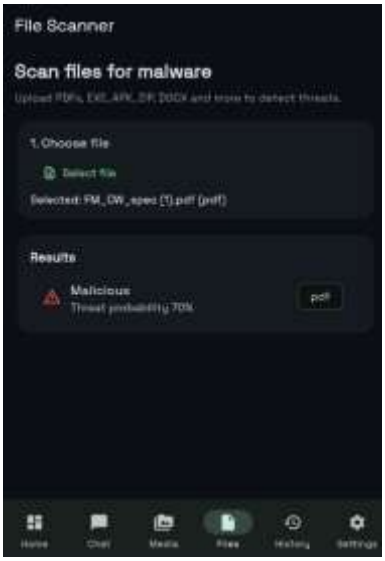
This is where you control everything about how the app works for you. You can change your name or email, turn notifications on or off (for alerts and weekly reports), read the privacy policy, and choose light mode, dark mode, or system default look. It helps you personalize and manage your account.



This screen shows the full list of every scan the app has ever done. You can see old results from many days ago. You can filter to show only chats, only media, only files, or everything. Each scan has a color icon (green = safe, orange = maybe problem, red = danger), the type of scan, date, and risk level. It works like a complete security diary.

File Scanner

This screen lets you check files for dangerous malware or viruses. You press "Select file" to upload things like PDFs, EXE programs, Word documents, and more. After uploading (example: FM_CW_spec(1).pdf), the app quickly scans it. If it finds something bad, it shows a big red warning "Malicious" with a threat probability percentage (like 70%). It keeps your files safe by detecting hidden threats.

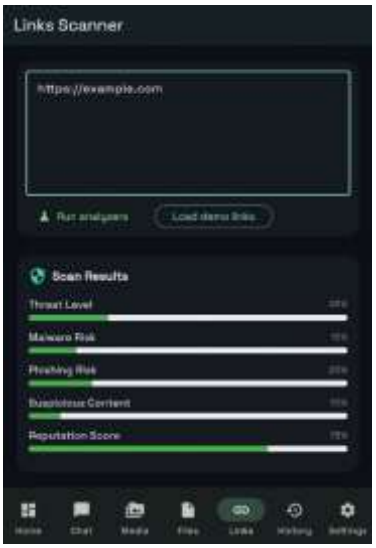


Chat Scanner

This screen analyzes text from WhatsApp, Telegram, or other chats for emotional tone and possible problems. You paste chat messages into the big box (example: conversation with John, Alice, Bob) or upload a chat file. Press "Run analyzers" to start. The app shows emotion bars like Joy (high = positive), Sadness, Anger, and Fear with percentages. It helps you spot if chats feel happy/healthy or if there is stress, manipulation, or negative emotions.

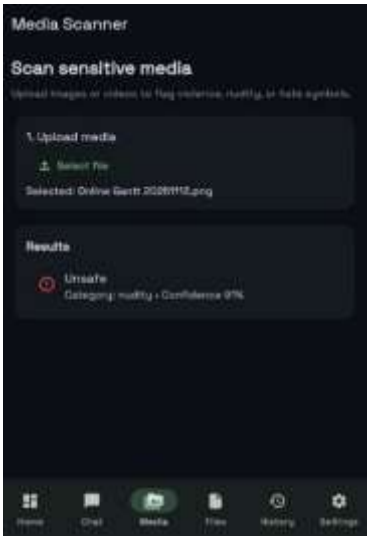
Links Scanner

This screen checks website links for danger before you visit them. You paste a link (example: https://example.com) into the box. Then press "Run analyzers" (or use demo links to test). The app gives a full report with bars showing: overall Threat Level (e.g., 25%), Malware Risk, Phishing Risk, Suspicious Content, and Reputation Score (higher is better). Green means mostly safe, red means high danger.



Media Scanner

This screen is for scanning photos, images, or videos to find sensitive or unsafe content. You click "Select file" to upload media (example: OnlineGantt20251113.png). The app checks for things like violence, nudity, or hate symbols. If it detects something, it shows a warning like "Unsafe" with the category (e.g., nudity) and a high confidence score (like 91%). It helps protect you from harmful or inappropriate visual content.



2.1.3 Hardware interfaces

The system does not require specialized hardware and operates on standard consumer mobile devices and server infrastructure.

- **Mobile Device Hardware**
    - Android smartphones with ARM-based processors
    - Minimum hardware components include CPU, RAM, storage, camera (for media access), and network interface (Wi-Fi or cellular)
    - Optional biometric hardware (fingerprint or face recognition) may be used for secure authentication
- **Server Hardware**
    - Cloud or on-premise servers with sufficient CPU, memory, and storage
    - Support for virtualization or containerization to host sandbox environments
    - Network interfaces supporting secure internet communication

No direct hardware coupling exists between client and server beyond standard networking equipment.

2.1.4 Software interfaces

The system depends on multiple software platforms and libraries for operation.

- **Mobile Platform**
  - Android OS (supported versions and above)
  - Android SDK and system services (notifications, background execution, file access)
- **Machine Learning Frameworks**
  - TensorFlow Lite for on-device inference
  - Server-side ML frameworks (e.g., TensorFlow / PyTorch) for advanced analysis
- **Backend Services**
  - RESTful APIs for secure communication
  - FLASK
  - Sandbox management software for isolated execution
  - Database systems for storing metadata, logs, and reports
- **Authentication Services**
  - Email/password authentication
  - Optional third-party authentication (e.g., Google Sign-In)

All software interfaces communicate through well-defined APIs with authentication and encryption enforced.

2.1.5 Communication interfaces

The system relies on secure communication mechanisms to exchange data between components.

- **Mobile-to-Server Communication**
  - HTTPS over TLS for encrypted data transmission
  - RESTful API calls for submitting extracted features and receiving analysis results
- **Server Internal Communication**
  - Secure inter-process communication between analysis modules and sandbox environments
  - Container or VM-based isolation networks
- **Notification Communication**
  - Android notification services for real-time alerts
  - Optional email or push notifications for parental summaries

No peer-to-peer communication between user devices is required.

2.1.6 Memory constraints

The system is designed to operate within the limited memory resources of mobile devices.

- On-device ML models shall be lightweight and optimized for low memory consumption.
- Temporary analysis data shall be cleared immediately after processing.
- Background services shall operate within OS-defined memory limits to prevent application termination.
- Server-side components may use higher memory resources but must scale efficiently to support concurrent users.

The application shall remain functional on mid-range Android devices without excessive memory usage.

2.1.7 Operations

This section describes the normal and exceptional operational activities performed by the system during regular use and maintenance. The operations are designed to be largely automated to minimize user involvement and reduce the risk of human error.

Normal Operations

- The system continuously operates in the background once enabled, monitoring user interactions with WhatsApp and Telegram without interrupting normal messaging behavior.
- When a user attempts to send or receive media files (images, videos, or documents), the system automatically initiates a pre-transmission scanning process to assess potential risks.
- URLs detected within messages are automatically extracted and analyzed using static feature analysis and sandbox-based behavioral inspection.
- Risk evaluation results are generated in real time and categorized into predefined severity levels (e.g., safe, suspicious, high risk).
- User notifications are displayed through non-intrusive alert mechanisms, allowing users to take informed actions without disrupting their workflow.
- All detection processes are executed efficiently to ensure low latency and minimal battery consumption.

Special Operations

- Parents or guardians can access a protected dashboard to review historical alerts, flagged content, and behavioral summaries.
- The system allows controlled override actions where authorized users may permit flagged content after reviewing associated risks.
- Model updates, threat database synchronization, and system configuration updates are handled automatically or during scheduled maintenance windows.
- Error handling mechanisms ensure system stability during unexpected failures, such as interrupted scans or insufficient system resources.

Overall, all operational activities are designed to be self-managed, fault-tolerant, and accessible to users with minimal technical expertise.

2.1.8 Site Adaptation Requirements

The system follows a hybrid architecture, combining mobile and server-based execution to ensure safety and scalability.

- The mobile application requires installation on Android devices running a supported OS version or higher.
- A secure backend server is required for:
  - Training machine learning models
  - Executing malicious URLs and documents within isolated sandbox environments
- The server environment must support virtualization or containerization technologies to safely contain malware.
- Stable network connectivity is required for server-assisted analysis of URLs and documents.
- The system supports execution in personal, institutional, or cloud-hosted environments without requiring user-side server setup.
- Device-specific UI customizations are handled dynamically by the mobile application.

This architecture ensures that malware never executes on the user's device, maintaining system safety.

2.2 Product functions

The proposed system delivers a comprehensive set of security and monitoring functions through a **coordinated interaction between mobile-side components and a secure backend server**. These functions are designed to operate seamlessly within encrypted messaging environments while prioritizing user safety, privacy, and system efficiency.

URL and Document Feature Handling

- The system securely extracts non-sensitive features from URLs and shared documents when they are detected within WhatsApp or Telegram interactions.
- Extracted features are transmitted to the backend server using encrypted communication channels to prevent unauthorized access or data leakage.
- Feature extraction is designed to avoid capturing personal message content, ensuring compliance with privacy requirements.

Server-Side Malware and URL Analysis

- Suspicious URLs are executed within a controlled server-side sandbox environment to observe redirection behavior, network communication patterns, and malicious payload delivery attempts.
- Shared documents (such as PDFs and office files) are dynamically analyzed in isolated sandbox instances to detect embedded exploits, macros, or malicious code.
- Machine learning models running on the server classify URLs and documents into predefined risk categories based on observed behavior and extracted features.

Threat Detection and Classification

- The system identifies phishing attempts, spam campaigns, and malware-hosting URLs using behavior-based and learning-based approaches.
- Documents are analyzed to detect known and unknown malware families, including obfuscated and zero-day threats.
- Risk levels are assigned to each analyzed artifact to support informed user decision-making.

On-Device Sensitive Media Detection

- Images and videos selected for sharing are analyzed locally using lightweight machine

learning models to detect nudity, explicit content, and sensitive personal information.

- On-device processing ensures real-time detection without transmitting private media to external servers.
- Users are warned before sensitive content is shared, reducing accidental exposure.

Chat Behavior and Emotional Analysis

- The system evaluates chat interaction patterns using privacy-aware analytical techniques to identify suspicious or manipulative behavior.
- Emotional cues and communication anomalies are assessed without storing or transmitting raw conversation content.
- Detected behavioral risks contribute to overall threat assessment and alert generation.

Alerting and User Notification

- Real-time alerts are generated based on combined results from server-side and on-device analyses.
- Alerts provide clear explanations of detected risks, severity levels, and recommended actions.
- Notifications are designed to be non-intrusive and easily understandable by non-technical users.

Parental Monitoring and Reporting

- A secure parental dashboard provides summarized insights into detected threats and user activity.
- Reports include trends, alert history, and categorized threat types rather than detailed content.
- Access to monitoring features is restricted to authorized users to maintain privacy and trust.

Privacy-Preserving Data Management

- The system minimizes data retention by avoiding permanent storage of malicious content or personal media on user devices.
- Server-side data is handled in accordance with privacy and ethical guidelines, with sandbox environments regularly reset.
- Only essential metadata and risk indicators are retained for reporting and system improvement purposes.

2.3 User characteristics

The proposed system is designed to support a **diverse group of users** with varying levels of technical expertise, cybersecurity awareness, and responsibility. The user interface, interaction flow, and system operations are tailored to ensure usability, clarity, and accessibility for all user categories.

Primary User Groups

Children and Teenagers

- Typically frequent users of WhatsApp and Telegram with high exposure to online content and communication risks.
- Possess limited awareness of cybersecurity threats such as phishing, malware, and social engineering.
- Require intuitive warnings, visual indicators, and simple explanations to understand potential risks.
- Interact with the system primarily through automatic alerts and guidance rather than manual configuration.

Parents or Guardians

- Have basic to moderate smartphone literacy but may lack specialized cybersecurity knowledge.
- Require summarized, non-technical insights into detected threats rather than raw analytical data.
- Use the system to monitor risk trends, review alerts, and guide safer online behavior.
- Expect straightforward controls, secure access mechanisms, and clear reporting.

General Users

- Represent everyday smartphone users seeking passive protection without active involvement.
- Prefer minimal system configuration and unobtrusive background operation.
- Rely on clear recommendations and simple action choices when threats are detected.
- May not be interested in detailed technical explanations.

Educators or Supervisors

- Responsible for overseeing digital safety in educational or controlled environments.

- Require aggregated and high-level visibility into safety trends rather than individual content.
- Use reports and summaries to promote awareness and digital literacy.
- Interact with the system occasionally rather than continuously.

User Skill Level and Training

- All user groups are considered **non-expert users** from a cybersecurity perspective.
- No advanced technical skills, prior training, or security configuration knowledge is required to operate the system.
- The system is designed to be **self-explanatory**, with guided actions and minimal user input.
- Default settings are optimized to provide effective protection without customization.

Overall, the system emphasizes **ease of use, clarity, and safety**, ensuring that users can benefit from advanced threat detection capabilities without needing technical expertise.

2.4 Constraints

The development, deployment, and operation of the proposed system are governed by several **technical, operational, legal, and environmental constraints**. These constraints limit certain design choices and influence system architecture, performance, and functionality.

Encryption and Platform Constraints

- End-to-end encryption employed by WhatsApp and Telegram prevents direct inspection of message content at the network or server level.
- The system must operate within OS-level accessibility and file-sharing mechanisms without modifying or bypassing messaging platform security.
- Direct integration with messaging platform APIs is not permitted, restricting the system to indirect interaction models.

Server-Side Security Constraints

- URLs and documents suspected of malicious behavior must be executed only within **isolated sandbox environments** hosted on secure backend servers.
- Strict containment mechanisms, including virtualization and process isolation, are required to prevent malware escape or lateral movement.
- Regular sandbox resets and controlled execution policies are mandatory to ensure long-term system safety.

Network and Communication Constraints

- Server-assisted analysis depends on stable internet connectivity.
- Network latency and bandwidth limitations may introduce delays in real-time threat detection.
- The system must handle intermittent connectivity gracefully by prioritizing on-device detection when server access is unavailable.

Mobile Device Resource Constraints

- Mobile devices impose limitations on CPU, memory, storage, and battery capacity.

- On-device machine learning models must remain lightweight to avoid degrading device performance.
- Background operations must comply with mobile OS restrictions on resource usage and power management.

Privacy and Regulatory Constraints

- Compliance with applicable data protection and privacy regulations restricts the collection, storage, and transmission of user data.
- Personal message content, images, and documents must not be permanently stored on user devices or backend servers.
- Only minimal metadata and risk indicators may be retained for reporting and system improvement.

Security and Policy Constraints

- All data exchanged between mobile and server components must be encrypted and authenticated.
- Access to parental dashboards and logs must be restricted through secure authentication mechanisms.
- Security policies must be consistently enforced across both mobile and server environments to prevent unauthorized access or misuse.

Overall, these constraints require the system to balance **security effectiveness, performance efficiency, and privacy preservation**, shaping a design that is both technically feasible and ethically compliant.

2.5 Assumptions and dependencies

This section identifies the **assumptions** made during the specification of system requirements and the **external dependencies** that influence the successful operation of the proposed system. These factors are not controlled directly by the system but have a significant impact on its functionality, performance, and reliability.

Assumptions

- Users will grant all required permissions, including access to media files, background services, and network communication, to enable complete system functionality.
- Users will not deliberately disable critical system services or permissions after installation.
- Secure backend server infrastructure will be continuously available to perform URL and document analysis tasks.
- Malware samples executed within the sandbox environment will remain fully isolated and will not escape or affect other systems.
- Messaging platforms will continue to allow OS-level accessibility and file-sharing mechanisms required for detection.
- Users possess basic smartphone operational knowledge, such as responding to notifications and following on-screen instructions.
- The volume and frequency of user interactions will remain within expected operational limits.

Dependencies

- The system depends on the availability, security, and scalability of backend server infrastructure for safe execution of malicious URLs and documents.
- Proper functioning of sandboxing, virtualization, and containerization technologies is required to ensure malware containment.
- Stable internet connectivity is necessary for server-assisted analysis, model updates, and threat intelligence synchronization.
- Continuous maintenance and updating of machine learning models and threat datasets are required to maintain detection accuracy.

- Compatibility with supported Android OS versions and ongoing OS updates is required.
- Availability of mobile ML inference libraries and secure communication protocols is required for seamless operation.

Failure to satisfy these assumptions or dependencies may result in reduced system performance, delayed threat detection, or partial functionality.

2.6 Apportioning of requirements

This section defines the **phased prioritization and implementation order** of system requirements. The apportioning strategy is designed to ensure early delivery of critical security functionality while managing technical risk, resource constraints, and system complexity.

Phase 1: Core Threat Detection Capabilities

- Implementation of server-based URL and document malware detection is prioritized due to the high risk posed by malicious links and files.
- Secure sandbox environments are established to safely execute and analyze potentially harmful content.
- Machine learning models for initial threat classification are integrated and validated.

Phase 2: Secure Communication Infrastructure

- Development of secure, encrypted communication channels between the mobile application and backend server.
- Implementation of authentication, authorization, and integrity checks for data exchange.
- Error handling and retry mechanisms to ensure reliable communication under varying network conditions.

Phase 3: On-Device Sensitive Media Detection

- Deployment of lightweight machine learning models for image and video analysis on mobile devices.
- Optimization of on-device inference to minimize performance impact and battery usage.
- Integration with user workflows to provide pre-transmission warnings.

Phase 4: Alerting and Notification Mechanisms

- Design and implementation of real-time alert generation based on combined server-side and on-device analysis results.
- Development of user-friendly notification formats with clear severity indicators and recommended actions.

- Validation of alert timing to ensure minimal disruption.

Phase 5: Parental Monitoring and Reporting Features

- Implementation of secure parental dashboards and access controls.
- Development of summarized reporting mechanisms, alert history views, and trend analysis.
- Ensuring privacy-preserving presentation of monitoring information.

Phase 6: Performance Optimization and Scalability

- Optimization of server-side processing to handle increasing analysis requests.
- Refinement of mobile resource usage, including memory, CPU, and battery consumption.
- Load testing and performance benchmarking under realistic usage scenarios.

Phase 7: Advanced Analytics and Future Enhancements

- Integration of advanced chat behavior and emotional analysis modules.
- Support for future expansion to additional messaging platforms and threat categories.
- Architectural refinements to support long-term maintainability and extensibility.

This phased approach ensures that essential security functionality is delivered early, while allowing incremental enhancement and risk-controlled expansion in later development stages.

3. Specific requirements(1) (for Software Dev. Oriented Projects - SRS)

3.1 External interface requirements

This section specifies the external interfaces through which users, hardware components, software systems, and communication mechanisms interact with the proposed multi-modal threat detection system.

3.1.1 User interfaces

- The system shall provide a **graphical mobile user interface (GUI)** designed for Android devices.

- The user interface shall be **simple, intuitive, and suitable for non-technical users**, including children and parents.

- The system shall include the following primary interfaces:

- User registration and login interface
    - Main dashboard displaying threat summaries and system status
    - Scanning interfaces for chats, media files, documents, and URLs
    - Scan history and detailed report views
    - Settings and account management interface

- Threat severity shall be communicated using **visual indicators** such as color codes, icons, and labels (e.g., Safe, Suspicious, High Risk).

- The interface shall support **real-time notifications and alerts** without disrupting normal device usage.

- The parental monitoring interface shall present **summarized insights** without exposing sensitive or private message content.

3.1.2 Hardware interfaces

- The system shall operate on **standard Android smartphones** without requiring specialized hardware.

- The application shall interface with mobile device hardware components including:

    - CPU and memory for on-device processing
    - Local storage for temporary file access
    - Network interface (Wi-Fi or cellular data) for server communication

- Optional hardware features such as **biometric authentication sensors** may be used for secure user access.

- No direct interaction with low-level hardware components shall be required beyond Android OS–provided APIs.

3.1.3 Software interfaces

- The system shall interface with the **Android operating system** using officially supported APIs and permission mechanisms.

- The application shall interface with a **secure backend server** through RESTful APIs for URL and document analysis.

- The system shall utilize **machine learning frameworks**:

    - TensorFlow Lite for on-device inference
    - Server-side ML frameworks for advanced behavioral analysis

- The system shall interface with:

    - Authentication services (email/password and optional third-party sign-in)
    - Notification services for alert delivery
    - Secure database systems for storing metadata and logs

- All software interfaces shall be modular and well-defined to support future extensibility.

3.1.4 Communication interface

- All communication between the mobile application and backend server shall use **secure encrypted protocols (HTTPS over TLS)**.
- The system shall support **REST-based communication** for sending extracted features and receiving analysis results.
- Communication payloads shall contain **only non-sensitive metadata and anonymized features**.
- The system shall handle **network latency, disconnections, and retries** gracefully.
- Authentication tokens and secure session management shall be used to prevent unauthorized access.
- No direct communication between user devices shall be required.

3.2 Classes/Objects < For Software Dev. Oriented Projects>

This section categorizes domain-level classes used to organize system functionality. Each class is marked by priority.

Class / Component	Description	Priority
URLAnalyzer	Extracts, sends, and evaluates URL threats	Essential
DocumentAnalyzer	Handles sandbox-based document malware analysis	Essential
MediaContentScanner	Detects sensitive images and videos	Essential
ChatBehaviorAnalyzer	Identifies emotional manipulation and fake behavior	Desirable
AlertManager	Generates and manages alerts and notifications	Essential
ParentalDashboard	Provides monitoring and reporting	Essential
ServerCommunicationManager	Handles secure data exchange	Essential
ModelUpdateManager	Updates ML models and datasets	Desirable
LoggingManager	Stores event logs securely	Optional

These classes are sufficient to organize system requirements but do not represent all implementation-level classes.

3.3 Performance requirements

This section defines the **quantitative and qualitative performance expectations** of the system under normal operating conditions. Performance requirements are specified to ensure that security functions do not negatively impact user experience or device stability.

Response Time Requirements

- URL threat analysis and document malware detection results shall be returned within acceptable response times under normal network conditions.
- Server-assisted analysis shall provide results quickly enough to support real-time user decision-making.
- In cases of network delay, the system shall notify the user and provide interim guidance where applicable.

On-Device Processing Performance

- Image and video content scanning shall complete within a short duration to avoid noticeable delays in the user workflow.
- On-device machine learning inference shall be optimized to ensure smooth application responsiveness.
- Temporary processing data shall be cleared immediately after analysis to reduce memory usage.

Resource Utilization

- Background monitoring processes shall consume minimal CPU resources and operate within defined memory limits.
- The system shall avoid excessive disk usage by limiting the size and retention of logs and cached data.
- Resource usage shall adapt dynamically based on device capabilities.

Battery Consumption

- Battery usage shall remain within acceptable limits during continuous background

operation.

- Energy-efficient scheduling and throttling mechanisms shall be employed to reduce power consumption.
- The system shall comply with mobile OS power management policies.

Server-Side Performance and Scalability

- The backend server shall support concurrent URL and document analysis requests without significant degradation in response time.
- Sandbox environments shall be efficiently managed to allow parallel execution and rapid reset.
- The system shall be scalable to handle increased workloads as the user base grows.

Performance Optimization

- Continuous performance monitoring and profiling shall be conducted to identify bottlenecks.
- Optimizations shall be applied without compromising detection accuracy or security.
- Performance improvements shall prioritize usability on mid-range mobile devices.

3.4 Design constraints

The design of the proposed system is governed by a set of technical, architectural, regulatory, and operational constraints that limit available design options. These constraints must be strictly adhered to in order to ensure system feasibility, security, compliance, and user trust.

Encryption and Platform Constraints

- End-to-end encryption enforced by WhatsApp and Telegram prevents direct inspection of message content at the network or server level.
- The system design must avoid any attempt to bypass, weaken, or interfere with platform encryption mechanisms.
- Detection logic must operate using OS-level accessibility, file-handling, and user-interaction cues rather than direct message interception.

Malware Safety Constraints

- Potentially malicious URLs and documents must never be executed on the user's device.
- All malware analysis must occur within secure, isolated server-side sandbox environments.
- The system design must include strict isolation, controlled execution, and automated sandbox reset mechanisms to prevent contamination.

Mobile Operating System Constraints

- The design must comply with Android OS permission models, background execution limits, and power management policies.
- Continuous background monitoring must be implemented using approved system services to avoid application termination.
- The system must gracefully handle permission denial or revocation by the user.

Machine Learning Constraints

- Machine learning models deployed on mobile devices must be lightweight and optimized for on-device inference.
- Model complexity must be balanced against device resource availability, including CPU, memory, and battery.
- Server-side models may be more computationally intensive but must remain scalable and secure.

Privacy and Regulatory Constraints

- The design must comply with applicable data protection and privacy regulations.
- Personal message content, images, and documents must not be permanently stored on either the mobile device or backend server.
- Only minimal metadata, anonymized features, and risk indicators may be retained for operational purposes.

Architectural Compliance

- Design decisions must align with a modular and extensible architecture.
- Future enhancements must be accommodated without violating existing constraints.
- Security, privacy, and performance requirements must be enforced consistently across mobile and server components.

Overall, these design constraints ensure that the system delivers effective threat detection while preserving user privacy, device safety, and regulatory compliance.

3.5 Software system attributes

3.5.1 Reliability

- The system shall operate continuously during normal device usage with minimal functional failures.
- Automatic error detection and recovery mechanisms shall be implemented to handle communication interruptions, analysis failures, or temporary server unavailability.
- Partial system failures shall not result in incorrect threat classification or unsafe user actions.
- In the event of component failure, the system shall degrade gracefully and notify the user when appropriate.
- System logs shall be maintained to support fault diagnosis and long-term reliability improvements.

3.5.2 Availability

- Core on-device threat detection features shall remain available whenever the mobile device is powered on and operational.
- Server-assisted analysis shall be accessible subject to network connectivity and backend availability.
- The system shall automatically resume normal operations after interruptions such as network loss, application restarts, or device reboots.
- Planned maintenance activities shall be performed in a manner that minimizes service disruption.
- Availability requirements shall support continuous protection during typical daily usage patterns.

3.5.3 Security

- All data exchanged between mobile and server components shall be encrypted using secure communication protocols.
- Malware execution and analysis shall be strictly confined to isolated sandbox environments to prevent system compromise.

- Authentication and authorization mechanisms shall restrict access to parental dashboards, logs, and configuration settings.

- Sensitive user data, including message content, images, and documents, shall not be permanently stored on mobile devices or backend servers.

- The system shall be designed to resist common security threats such as unauthorized access, data leakage, and tampering.

3.5.4 Maintainability

- The system shall follow a modular and loosely coupled architecture to facilitate updates, enhancements, and bug fixes.

- Machine learning models, detection rules, and threat datasets shall be updateable without requiring full application reinstallation.

- Clear and structured logging shall be implemented to support debugging, performance monitoring, and maintenance activities.

- Documentation shall be maintained to support developers and future maintainers.

- The system shall be designed to accommodate future feature additions with minimal impact on existing components

3.6 Other Requirements

This section captures additional system requirements that do not fall directly under functional, performance, or core quality attributes but are essential for the system's long-term usability, compliance, and extensibility.

Privacy and Ethical Compliance

- The system shall comply with applicable privacy, data protection, and ethical standards governing the collection, processing, and storage of user data.
- User consent shall be respected, and data handling practices shall be transparent and non-intrusive.
- The system shall avoid intrusive monitoring and shall support user autonomy and informed decision-making.
- Ethical considerations related to child safety and parental monitoring shall be carefully balanced with individual privacy rights.

Extensibility and Future Integration

- The system shall be designed to support future integration with additional messaging platforms without requiring significant architectural changes.
- Detection modules shall be modular to allow the addition of new threat categories or analytical techniques.
- Server-side components shall be scalable to accommodate future expansion.

Localization and Accessibility

- The system shall support future localization to accommodate multiple languages and regional requirements.
- User interfaces shall be designed to allow accessibility enhancements, such as readable text sizes, clear icons, and compatibility with assistive technologies.
- Accessibility improvements shall aim to make the system usable by individuals with varying abilities.

Auditing and Monitoring

- The system shall generate audit logs to record significant events such as threat detections, alerts, and system updates.
- Audit logs shall be protected against unauthorized access and tampering.
- Logged information shall be limited to metadata and risk indicators, avoiding storage of sensitive user content.
- Audit logs shall support system evaluation, performance analysis, and compliance verification.

4. References

[1] I. Sommerville, Software Engineering, 10th ed. Boston, MA, USA: Pearson, 2016.

[2] IEEE Computer Society, IEEE Standard for Software Requirements Specifications, IEEE Std 830-1998, 1998.

[3] ISO/IEC/IEEE, Systems and Software Engineering—Life Cycle Processes—Requirements Engineering, ISO/IEC/IEEE 29148:2018, 2018.

[4] M. Christodorescu and S. Jha, "Static analysis of executables to detect malicious patterns," in Proc. 12th USENIX Security Symp., Washington, DC, USA, 2003, pp. 169–186.

[5] C. Willems, T. Holz, and F. Freiling, "Toward automated dynamic malware analysis using CWSandbox," IEEE Security & Privacy, vol. 5, no. 2, pp. 32–39, Mar.–Apr. 2007.

[6] A. Shabtai, U. Kanonov, Y. Elovici, C. Glezer, and Y. Weiss, "Andromaly: A behavioral malware detection framework for Android devices," Journal of Intelligent Information Systems, vol. 38, no. 1, pp. 161–190, 2012.

[7] K. Rieck, T. Holz, C. Willems, P. Düssel, and P. Laskov, "Learning and classification of malware behavior," in Proc. Detection of Intrusions and Malware & Vulnerability Assessment (DIMVA), Paris, France, 2008, pp. 108–125.

[8] M. Egele, T. Scholte, E. Kirda, and C. Kruegel, "A survey on automated dynamic malware-analysis techniques and tools," ACM Computing Surveys, vol. 44, no. 2, pp. 1–42, 2012.

[9] J. Ma, L. K. Saul, S. Savage, and G. M. Voelker, "Beyond blacklists: Learning to detect malicious web sites from suspicious URLs," in Proc. 15th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, Paris, France, 2009, pp. 1245–1254.

[10] Y. Zhang, J. Zhang, and Z. Wang, "Detecting malicious URLs via machine learning based approaches," IEEE Access, vol. 7, pp. 140260–140275, 2019.

[11] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," Advances in Neural Information Processing Systems (NeurIPS), vol. 25, pp. 1097–1105, 2012.

[12] S. Zhai, K. Zhang, and H. Wang, "Privacy-preserving machine learning in mobile environments," IEEE Communications Surveys & Tutorials, vol. 22, no. 3, pp. 1778–1802, 2020.

[13] Google Developers, "TensorFlow Lite: Machine learning for mobile and embedded devices," Google, Mountain View, CA, USA, 2023.

[14] N. Milosevic, A. Dehghantanha, and K.-K. R. Choo, "Machine learning aided Android malware classification," Computers & Security, vol. 80, pp. 266–274, 2019.

[15] E. Kirda, C. Kruegel, G. Banks, G. Vigna, and R. Kemmerer, "Behavior-based spyware detection," in Proc. 15th USENIX Security Symp., Vancouver, BC, Canada, 2006, pp. 273–288.